

Algorithmes Génétiques : Comprendre la Métaheuristique Évolutionniste

Du Concept aux Applications Pratiques

Résumé

Ce cours explique les algorithmes génétiques d'une manière intuitive, en partant de la biologie évolutionniste pour arriver aux applications pratiques. Nous déconstruisons les concepts complexes en analogies simples, avant de montrer comment résoudre des problèmes réels comme l'emploi du temps, le voyageur de commerce, et l'optimisation de paramètres.

Table des matières

1	Introduction : Pourquoi les Algos Génétiques sont Différents	2
1.1	Le Paradigme Évolutionniste	2
1.2	Analogie Biologique	3
2	Les 5 Composants Fondamentaux	3
2.1	1. Représentation (Chromosome)	3
2.2	2. Fonction Fitness	4
2.3	3. Sélection	4
2.4	4. Croisement (Crossover)	5
2.5	5. Mutation	5
3	Le Cycle de Vie d'un Algorithme Génétique	6
4	Exemple 1 : Emploi du temps Universitaire	6
4.1	Problématique	6
4.2	Représentation	7
4.3	Fonction Fitness	7
4.4	Croisement et Mutation	7
4.5	Exécution Complète	8
5	Exemple 2 : Le Voyageur de Commerce (TSP)	9
5.1	Problématique	9
5.2	Représentation pour TSP	9
5.3	Croisement spécial pour TSP	10
5.4	Mutation pour TSP	11

6	Exemple 3 : Optimisation de Fonctions Mathématiques	11
6.1	Problématique	11
6.2	Représentation pour optimisation continue	12
7	Paramètres et Bonnes Pratiques	12
7.1	Comment Choisir les Paramètres	12
7.2	Problèmes Courants et Solutions	13
7.3	Quand Utiliser un Algorithme Génétique	13
8	Implémentation Complète en Python	13
9	Applications Réelles	16
9.1	Industrie	16
9.2	Recherche	16
9.3	Exemple Industriel : Planification de Production	16
10	Conclusion	17
10.1	Ce qu'il faut retenir	17
10.2	Prochaines Étapes	17
10.3	Ressources pour Aller Plus Loin	17

1 Introduction : Pourquoi les Algos Génétiques sont Différents

1.1 Le Paradigme Évolutionniste

Concept Clé

Idée Fondamentale : Les algorithmes génétiques ne cherchent PAS la solution optimale directement. Ils font **évoluer** une population de solutions potentielles, en imitant la sélection naturelle.

Contrairement aux algorithmes traditionnels :

- **Algos classiques :** "Comment résoudre ce problème?"
- **Algos génétiques :** "Quelles solutions pourraient résoudre ce problème? Faisons-les se reproduire et muter pour trouver de meilleures solutions!"

1.2 Analogie Biologique

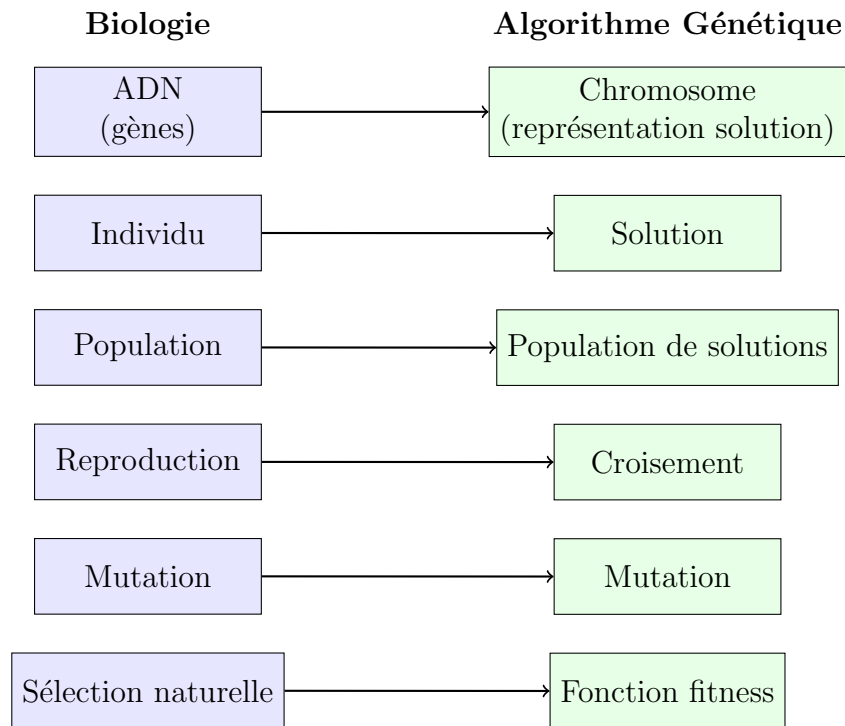


FIGURE 1 – Correspondance entre biologie et algorithmes génétiques

2 Les 5 Composants Fondamentaux

2.1 1. Représentation (Chromosome)

Concept Clé

Le **chromosome** est la façon dont on encode une solution dans un format que l'algorithme peut manipuler.

Types de représentations courantes :

Type	Exemple	Utilisation
Binaire	10110010	Problèmes simples, choix ON/OFF
Entier	[3,1,4,2,5]	Ordonnancement, permutations
Réel	[0.5, -2.3, 7.8]	Optimisation continue
Arbre		Programmation génétique

Exemple concret - Emploi du temps :

```
1 # Un chromosome pourrait être une liste de cours avec leurs horaires
2 # Chaque cours: (matière, professeur, salle, créneau)
3 chromosome = [
4     ("Maths", "Prof1", "A101", "Lundi 9h"),
5     ("Physique", "Prof2", "B202", "Lundi 10h"),
6     ("Chimie", "Prof3", "C303", "Mardi 9h")
7 ]
```

2.2 2. Fonction Fitness

Concept Clé

Le **fitness** évalue à quel point une solution est bonne. C'est l'équivalent de "la survie du plus apte".

Caractéristiques d'une bonne fonction fitness :

- Doit être **calculable** (pas besoin d'être parfaite)
- Doit **distinguer** les bonnes solutions des mauvaises
- Peut être **multi-objectif** (plusieurs critères)

Exemple emploi du temps :

```
1 def fitness(emploi_du_temps):
2     score = 100 # Score initial
3
4     # Penalités pour les problèmes
5     for conflit in detecter_conflits(emploi_du_temps):
6         score -= 10 # -10 points par conflit
7
8     for prof_surcharge in detecter_surcharge(emploi_du_temps):
9         score -= 5 # -5 points par prof surchargé
10
11     # Bonus pour les bonnes choses
12     if emploi_du_temps_respecte_preferences(emploi_du_temps):
13         score += 20
14
15     return max(0, score) # Score minimum de 0
```

2.3 3. Sélection

Concept Clé

La **sélection** choisit quels individus se reproduisent. Les meilleurs (fitness élevé) ont plus de chances.

Méthodes de sélection courantes :

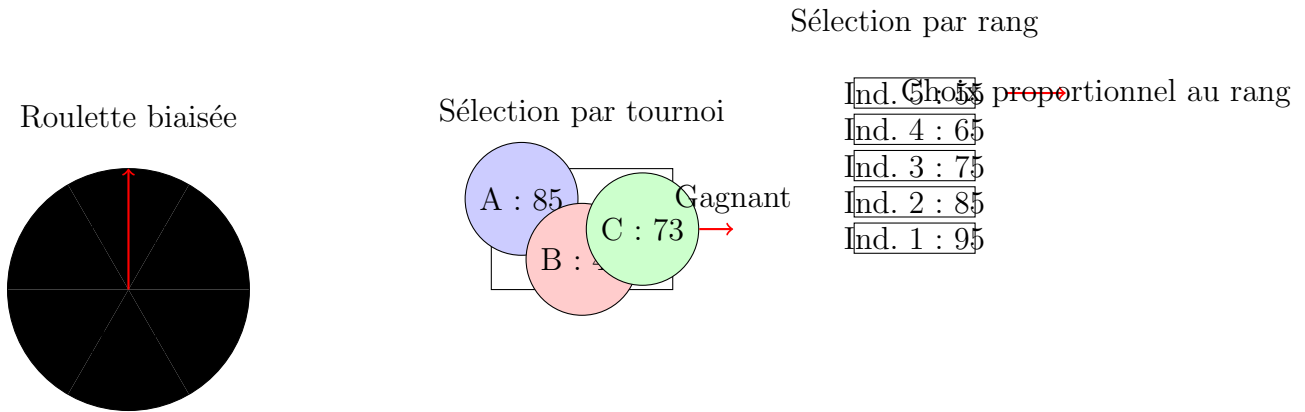


FIGURE 2 – Différentes méthodes de sélection

2.4 4. Croisement (Crossover)

Concept Clé

Le croisement combine deux parents pour créer des enfants. C'est le cœur de l'innovation dans l'algorithme.

Types de croisement :

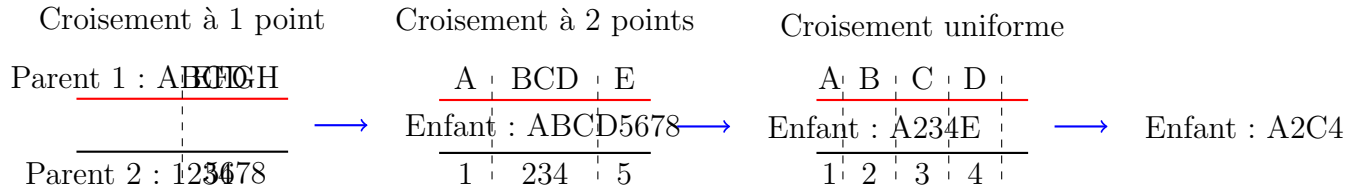


FIGURE 3 – Différents types de croisement

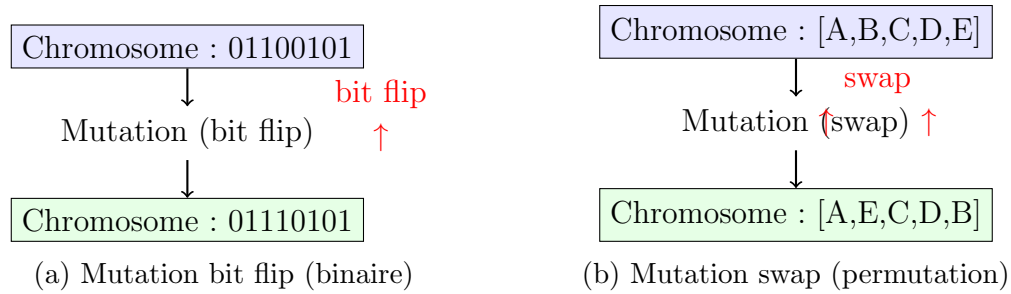
2.5 5. Mutation

Concept Clé

La mutation introduit de petites modifications aléatoires. Elle empêche la convergence prématurée et explore de nouvelles zones.

Taux de mutation :

- Trop faible → Pas d'exploration
- Trop élevé → Recherche aléatoire
- Typique : 1% à 10%



3 Le Cycle de Vie d'un Algorithme Génétique

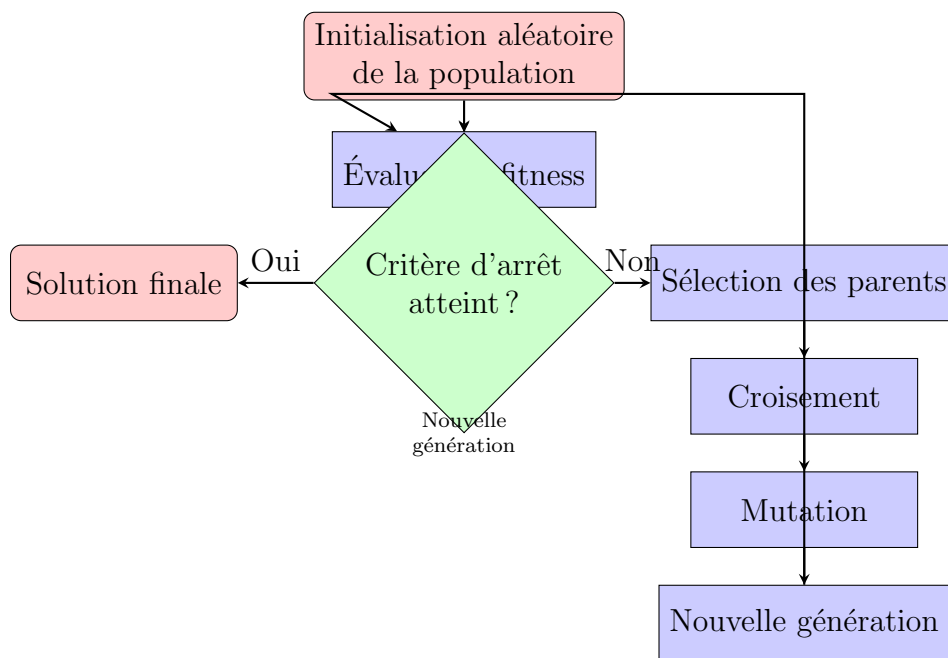


FIGURE 5 – Cycle complet d'un algorithme génétique

4 Exemple 1 : Emploi du temps Universitaire

4.1 Problématique

Données :

- 30 cours à planifier
- 10 salles disponibles
- 20 professeurs
- 5 créneaux horaires par jour, 5 jours par semaine
- Contraintes :
 - Pas de cours en même temps pour un même prof

- Capacité salle nombre d'étudiants
- Certains cours nécessitent des salles spéciales
- Préférences des professeurs

4.2 Représentation

```

1 # Une solution = un chromosome
2 # Chaque gène = (cours_id, salle_id, créneau_id, jour_id)
3
4 chromosome = [
5     # (cours, salle, créneau, jour)
6     (0, 3, 2, 1), # Cours 0 en salle 3, créneau 2, mardi
7     (1, 5, 4, 3), # Cours 1 en salle 5, créneau 4, jeudi
8     (2, 2, 1, 0), # Cours 2 en salle 2, créneau 1, lundi
9     # ... 27 autres cours
10 ]

```

4.3 Fonction Fitness

```

1 def fitness_emploi_du_temps(chromosome):
2     score = 1000 # Score maximum théorique
3
4     # 1. Penalités pour les conflits graves
5     conflits = detecter_conflits(chromosome)
6     score -= len(conflits) * 50 # -50 par conflit
7
8     # 2. Salles trop petites
9     for cours, salle, creneau, jour in chromosome:
10         if capacite_salle[salle] < etudiants_par_cours[cours]:
11             score -= 30
12
13     # 3. Préférences des professeurs
14     for prof in professeurs:
15         emploi = emploi_du_prof(prof, chromosome)
16         if emploi_trop_charge(emploi):
17             score -= 20
18         if respecte_preferences(prof, emploi):
19             score += 10
20
21     # 4. Répartition équilibrée
22     if repartition_equilibree(chromosome):
23         score += 50
24
25     return max(0, score) # Score minimum 0

```

4.4 Croisement et Mutation

```

1 def crossover_emploi_du_temps(parent1, parent2):
2     """Croisement un point pour l'emploi du temps"""
3     point = random.randint(1, len(parent1)-1)
4     enfant = parent1[:point] + parent2[point:]
5     return enfant
6
7 def mutation_emploi_du_temps(chromosome, taux_mutation=0.1):
8     """Mutation : changer alatoirement salle ou creneau"""
9     for i in range(len(chromosome)):
10         if random.random() < taux_mutation:
11             cours, salle, creneau, jour = chromosome[i]
12
13             # Mutation type 1 : changer salle
14             if random.random() < 0.5:
15                 nouvelle_salle = random.choice(salles_disponibles)
16                 chromosome[i] = (cours, nouvelle_salle, creneau, jour)
17
18             # Mutation type 2 : changer creneau
19             else:
20                 nouveau_creneau = random.randint(0, 4)
21                 chromosome[i] = (cours, salle, nouveau_creneau, jour)
22
23     return chromosome

```

4.5 Exécution Complète

```

1 def algorithme_genetique_emploi_du_temps():
2     # 1. Initialisation
3     population = [generer_emploi_aleatoire() for _ in range(100)]
4
5     meilleur_score = 0
6     meilleure_solution = None
7
8     for generation in range(500): # 500 generations max
9         # 2. valuation
10         scores = [fitness_emploi_du_temps(ind) for ind in population]
11
12         # 3. Sauvegarde du meilleur
13         max_score = max(scores)
14         if max_score > meilleur_score:
15             meilleur_score = max_score
16             meilleure_solution = population[scores.index(max_score)]
17
18         # 4. Condition d'arrêt
19         if generation > 50 and pas_damelioration(derniers_scores):
20             break
21
22         # 5. Sélection
23         parents = selection_par_tournoi(population, scores, k=50)
24
25         # 6. Reproduction

```

```

26     enfants = []
27     while len(enfants) < len(population):
28         parent1, parent2 = random.sample(parents, 2)
29         enfant = crossover_emploi_du_temps(parent1, parent2)
30         enfant = mutation_emploi_du_temps(enfant, taux_mutation=0.05)
31         enfants.append(enfant)
32
33     # 7. Nouvelle g n ration ( elitisme : garde les 5% meilleurs)
34     population = enfants
35     garder_meilleurs(population, scores, top_percent=0.05)
36
37     return meilleure_solution, meilleur_score

```

5 Exemple 2 : Le Voyageur de Commerce (TSP)

5.1 Probl matique

Concept Cl 

Probl me : Trouver le chemin le plus court qui visite toutes les villes exactement une fois et revient   la ville de d part.

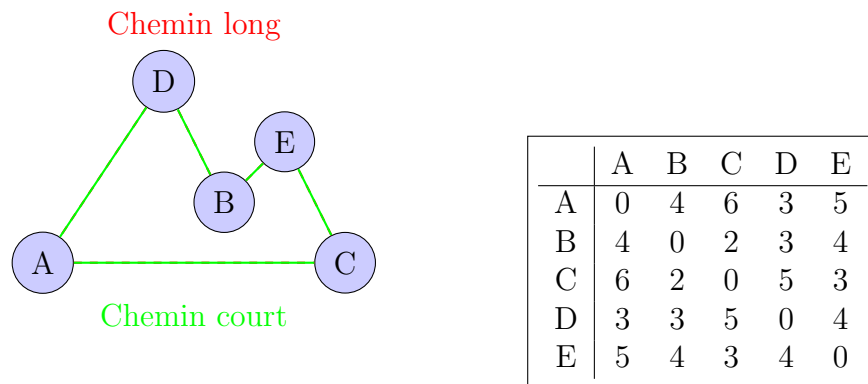


FIGURE 6 – Probl me du voyageur de commerce avec 5 villes

5.2 Repr sentation pour TSP

```

1 # Solution = permutation des villes
2 # Exemple: [0, 3, 1, 4, 2] signifie:
3 # A      D      B      E      C      A
4
5 chromosome = [0, 3, 1, 4, 2] # Indices des villes
6
7 # Pour calculer la distance totale:
8 def distance_totale(permutation):
9     total = 0

```

```

10     n = len(permutation)
11     for i in range(n):
12         ville_actuelle = permutation[i]
13         ville_suivante = permutation[(i + 1) % n] # Retour au d part
14         total += distances[ville_actuelle][ville_suivante]
15     return total
16
17 # Fitness (on veut minimiser la distance)
18 def fitness_tsp(chromosome):
19     distance = distance_totale(chromosome)
20     # Plus la distance est petite, plus le fitness est grand
21     return 1000 / (1 + distance) # Transformation

```

5.3 Croisement spécial pour TSP

```

1 def crossover_0X(parent1, parent2):
2     """
3     Croisement par ordre (Order Crossover) pour TSP
4     Pr serve l'ordre relatif des villes
5     """
6     size = len(parent1)
7
8     # Choisir deux points de coupure
9     point1 = random.randint(0, size - 1)
10    point2 = random.randint(point1 + 1, size)
11
12    # Initialiser enfant avec des -1
13    enfant = [-1] * size
14
15    # Copier le segment du parent1
16    enfant[point1:point2] = parent1[point1:point2]
17
18    # Remplir le reste avec l'ordre du parent2
19    parent2_index = 0
20    enfant_index = 0
21
22    while enfant_index < size:
23        # Si on est dans le segment copi , sauter
24        if point1 <= enfant_index < point2:
25            enfant_index += (point2 - point1)
26            continue
27
28        # Prendre le prochain g ne du parent2 qui n'est pas d j dans l
29        'enfant
30        while parent2[parent2_index] in enfant:
31            parent2_index += 1
32
33        enfant[enfant_index] = parent2[parent2_index]
34        enfant_index += 1
35        parent2_index += 1
36
37    return enfant

```

```

37
38 # Exemple:
39 parent1 = [0, 1, 2, 3, 4, 5]
40 parent2 = [3, 5, 0, 1, 4, 2]
41 point1 = 2, point2 = 4
42 enfant = [?, ?, 2, 3, ?, ?] # Copie segment
43 # Remplissage: 3,5,0,1,4,2 dans l'ordre, sauf 2,3 d j pr sents
44 # R sultat: [0, 5, 2, 3, 1, 4]

```

5.4 Mutation pour TSP

```

1 def mutation_swap(chromosome, taux_mutation=0.1):
2     """Mutation par change de deux villes"""
3     if random.random() < taux_mutation:
4         i, j = random.sample(range(len(chromosome)), 2)
5         chromosome[i], chromosome[j] = chromosome[j], chromosome[i]
6     return chromosome
7
8 def mutation_inversion(chromosome, taux_mutation=0.1):
9     """Mutation par inversion d'un segment"""
10    if random.random() < taux_mutation:
11        i = random.randint(0, len(chromosome) - 2)
12        j = random.randint(i + 1, len(chromosome) - 1)
13        chromosome[i:j] = reversed(chromosome[i:j])
14    return chromosome

```

6 Exemple 3 : Optimisation de Fonctions Mathématiques

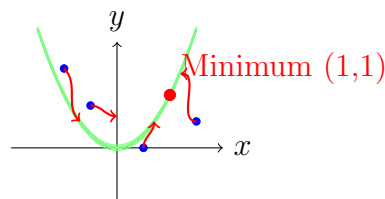
6.1 Problématique

Concept Clé

Objectif : Trouver le minimum global d'une fonction complexe avec plusieurs minima locaux.

Fonction test classique : Rosenbrock

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2$$



Fonction de Rosenbrock - "Banana Valley"

FIGURE 7 – Optimisation de la fonction de Rosenbrock

6.2 Représentation pour optimisation continue

```
1 # Solution = vecteur de nombres r els
2 # Exemple: [x, y] pour Rosenbrock
3
4 chromosome = [0.5, -0.3] # Coordonn es (x, y)
5
6 # Fitness pour minimisation
7 def fitness_rosenbrock(chromosome):
8     x, y = chromosome[0], chromosome[1]
9     # Fonction de Rosenbrock
10    value = (1 - x)**2 + 100 * (y - x**2)**2
11
12    # Plus la valeur est petite, plus le fitness est grand
13    return 1 / (1 + value)
14
15 # Croisement arithm tique
16 def crossover_arithmetic(parent1, parent2, alpha=0.5):
17     """Croisement par combinaison lin aire"""
18     enfant = []
19     for i in range(len(parent1)):
20         # Enfant = alpha*parent1 + (1-alpha)*parent2
21         gene = alpha * parent1[i] + (1 - alpha) * parent2[i]
22         enfant.append(gene)
23     return enfant
24
25 # Mutation gaussienne
26 def mutation_gaussian(chromosome, taux_mutation=0.1, sigma=0.1):
27     """Mutation avec bruit gaussien"""
28     for i in range(len(chromosome)):
29         if random.random() < taux_mutation:
30             chromosome[i] += random.gauss(0, sigma)
31     return chromosome
```

7 Paramètres et Bonnes Pratiques

7.1 Comment Choisir les Paramètres

Paramètre	Valeurs typiques	Conseils
Taille population	50-500	Plus grand = plus d'exploration
Taux mutation	1%-10%	Commencez par 5%
Taux croisement	70%-95%	Souvent 80%
Nombre générations	100-5000	Dépend de la complexité
Élitisme	1%-10%	Gardez les meilleurs

7.2 Problèmes Courants et Solutions

Problème	Solution
Convergence prématurée	Augmenter le taux de mutation, diversité initiale
Stagnation	Changer les opérateurs, adaptive mutation
Temps de calcul long	Réduire la population, fitness approximative
Mauvaises solutions	Revoir la fonction fitness

7.3 Quand Utiliser un Algorithme Génétique

Bons cas d'usage :

- Espace de recherche très grand
- Plusieurs optimums locaux
- Fonction objectif non différentiable
- Contraintes complexes
- Solutions "bonnes" acceptables (pas besoin d'optimal)

Mauvais cas d'usage :

- Solution optimale nécessaire
- Espace de recherche petit (force brute mieux)
- Fonction objectif simple et convexe
- Contraintes linéaires (simplex mieux)

8 Implémentation Complète en Python

```
1 import random
2 from typing import List, Callable, Tuple
3
4 class GeneticAlgorithm:
5     """Framework générique pour algorithmes génétiques"""
6
7     def __init__(self,
8                 population_size: int = 100,
9                 mutation_rate: float = 0.05,
10                 crossover_rate: float = 0.8,
11                 elitism: float = 0.1):
12         self.population_size = population_size
13         self.mutation_rate = mutation_rate
14         self.crossover_rate = crossover_rate
15         self.elitism = elitism
16
```

```

17     def run(self,
18             create_individual: Callable,
19             fitness_function: Callable,
20             crossover_function: Callable,
21             mutation_function: Callable,
22             max_generations: int = 500,
23             target_fitness: float = None):
24         """
25         Exécute l'algorithme génétique.
26
27         Args:
28             create_individual: Fonction qui crée un individu aléatoire
29             fitness_function: Fonction qui évalue un individu
30             crossover_function: Fonction de croisement
31             mutation_function: Fonction de mutation
32             max_generations: Nombre maximum de générations
33             target_fitness: Fitness cible pour arrêter prématurément
34         """
35
36         # 1. Initialisation
37         population = [create_individual() for _ in range(self.
38 population_size)]
39         best_individual = None
40         best_fitness = -float('inf')
41         history = []
42
43         for generation in range(max_generations):
44             # 2. valuation
45             fitnesses = [fitness_function(ind) for ind in population]
46
47             # 3. Sauvegarde du meilleur
48             max_fitness = max(fitnesses)
49             if max_fitness > best_fitness:
50                 best_fitness = max_fitness
51                 best_individual = population[fitnesses.index(max_fitness)]
52
53             history.append({
54                 'generation': generation,
55                 'best_fitness': best_fitness,
56                 'avg_fitness': sum(fitnesses) / len(fitnesses),
57                 'best_individual': best_individual.copy() if hasattr(
58 best_individual, 'copy') else best_individual
59             })
60
61             # 4. Condition d'arrêt
62             if target_fitness and best_fitness >= target_fitness:
63                 print(f"Solution trouvée à la génération {generation}")
64                 break
65
66             # 5. Sélection par roulette biaisée
67             selected = self._selection(population, fitnesses)
68
69             # 6. Reproduction

```

```

68         next_generation = []
69
70         # elitisme : garder les meilleurs
71         elite_count = int(self.elitism * self.population_size)
72         elite_indices = sorted(range(len(fitnesses)),
73                                key=lambda i: fitnesses[i],
74                                reverse=True)[:elite_count]
75         for idx in elite_indices:
76             next_generation.append(population[idx])
77
78         # Créer le reste de la population
79         while len(next_generation) < self.population_size:
80             if random.random() < self.crossover_rate:
81                 parent1, parent2 = random.sample(selected, 2)
82                 child = crossover_function(parent1, parent2)
83             else:
84                 child = random.choice(selected)
85
86             child = mutation_function(child, self.mutation_rate)
87             next_generation.append(child)
88
89         population = next_generation[:self.population_size]
90
91         # Affichage de progression
92         if generation % 50 == 0:
93             print(f"Generation {generation}: Best fitness = {
best_fitness:.4f}")
94
95         return best_individual, best_fitness, history
96
97     def _selection(self, population: List, fitnesses: List[float]) -> List
:
98         """Sélection par roulette biaisée"""
99         total_fitness = sum(fitnesses)
100         if total_fitness == 0:
101             # Si tous les fitness sont 0, sélection uniforme
102             return random.choices(population, k=len(population))
103
104         probabilities = [f / total_fitness for f in fitnesses]
105         return random.choices(population, weights=probabilities, k=len(
population))
106
107 # Exemple d'utilisation pour un problème simple
108 if __name__ == "__main__":
109     # Problème: Maximiser la somme des bits dans un chromosome binaire
110     def create_individual():
111         return [random.randint(0, 1) for _ in range(20)]
112
113     def fitness(individual):
114         return sum(individual) # Plus de 1 = mieux
115
116     def crossover(parent1, parent2):
117         point = random.randint(1, len(parent1)-1)
118         return parent1[:point] + parent2[point:]

```

```

119
120     def mutation(individual, rate):
121         for i in range(len(individual)):
122             if random.random() < rate:
123                 individual[i] = 1 - individual[i] # Flip bit
124         return individual
125
126     # Execution
127     ga = GeneticAlgorithm(population_size=50, mutation_rate=0.02)
128     best, fitness_val, history = ga.run(
129         create_individual=create_individual,
130         fitness_function=fitness,
131         crossover_function=crossover,
132         mutation_function=mutation,
133         max_generations=100
134     )
135
136     print(f"Meilleure solution: {best}")
137     print(f"Fitness: {fitness_val}/20")

```

Listing 1 – Framework simple d’algorithme génétique

9 Applications Réelles

9.1 Industrie

- **Logistique** : Optimisation de tournées de livraison
- **Manufacturing** : Ordonnancement de production
- **Finance** : Optimisation de portefeuille
- **Design** : Conception de circuits électroniques

9.2 Recherche

- **Bio-informatique** : Alignement de séquences ADN
- **Neuroscience** : Entraînement de réseaux de neurones
- **Robotique** : Apprentissage de comportements
- **Art** : Génération d’images et musique

9.3 Exemple Industriel : Planification de Production

```

1 # Planification d’atelier (Job Shop Scheduling)
2 # Objectif: Minimiser le temps total de production
3
4 def planifier_production_avec_AG():
5     """
6     Planifie les opérations d’un atelier avec plusieurs machines
7     Chaque job a plusieurs opérations avec des temps spécifiques

```

```

8      """
9
10     # Repr sentation: Liste des jobs dans l'ordre d'ex cution
11     # [job1, job2, job3, job1, job3, job2...]
12
13     # Fitness: Temps total pour terminer tous les jobs
14     # (plus court = mieux)
15
16     # Croisement: Pr server les s quences valides
17     # Mutation:  changer  deux op rations
18
19     return solution_optimale

```

10 Conclusion

10.1 Ce qu'il faut retenir

Concept Clé

Les algorithmes génétiques ne sont PAS :

- Une solution magique à tous les problèmes
- Garantis de trouver l'optimum global
- Toujours plus rapides que d'autres méthodes

Les algorithmes génétiques SONT :

- Excellents pour les problèmes complexes avec beaucoup de contraintes
- Robuste face aux minima locaux
- Flexibles (s'adaptent à presque n'importe quel problème)
- Inspirants (basés sur des principes biologiques éprouvés)

10.2 Prochaines Étapes

1. **Pratiquez** avec des problèmes simples
2. **Expérimentez** avec différents opérateurs
3. **Visualisez** l'évolution de la population
4. **Comparez** avec d'autres métaheuristiques
5. **Appliquez** à un problème réel qui vous intéresse

10.3 Ressources pour Aller Plus Loin

- **Livre** : "Genetic Algorithms in Search, Optimization, and Machine Learning" - David Goldberg
- **Cours** : MIT OpenCourseWare - "Introduction to Algorithms"

- **Bibliothèque** : DEAP (Distributed Evolutionary Algorithms in Python)
- **Compétition** : Google Hash Code (problèmes d'optimisation)

Le plus important : Comprendre que les AG sont un outil dans la boîte à outils de l'optimisation, pas une solution universelle. Leur force vient de leur flexibilité et de leur capacité à explorer des espaces de solutions complexes de manière créative.